

Machine Learning

Estimators, Transformers, and Predictors

Scikit-Learn uses a consistent **API design** for Machine Learning workflows. Three important concepts in this API are **Estimators, Transformers, and Predictors**.

1. Estimators

An **Estimator** is an object that can **learn parameters or information from data**.

In Scikit-Learn, an estimator generally provides a:

`fit()`

method.

The `fit()` method is used to learn something from the data.

Example

Consider a `StandardScaler`:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
scaler.fit(X_train)
```

When `fit()` is called, the imputer examines `X_train` and learns the relevant feature.

Other examples of Estimators

Many Scikit-Learn objects are estimators, including:

- `SimpleImputer`
- `StandardScaler`
- `LinearRegression`
- `DecisionTreeClassifier`

- KNeighborsClassifier

They are all capable of learning something from data through fit().

2. Transformers

A **Transformer** is an estimator that can **transform data** after learning the required information.

A transformer generally provides:

fit()

transform()

It can also usually provide:

fit_transform()

Example: SimpleImputer

```
imputer.fit(X_train)
```

```
X_train_transformed = imputer.transform(X_train)
```

Here:

- fit() → learns the replacement values.
- transform() → uses those learned values to modify the data.

So SimpleImputer is both:

Estimator + Transformer

Example: StandardScaler

A StandardScaler can learn the mean and standard deviation of each feature:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
scaler.fit(X_train)
```

It can then transform the data:

```
X_train_scaled = scaler.transform(X_train)
```

The scaler has therefore:

1. **Learned** parameters from the data.
2. **Transformed** the data using those parameters.

3. Predictors

A **Predictor** is an estimator that can **make predictions** based on learned patterns.

Predictors generally provide:

`fit()`

`predict()`

For example, consider a classification model:

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X_train, y_train)
```

The model learns patterns from the training data.

After training, it can make predictions:

```
predictions = model.predict(X_test)
```

Examples of predictors include:

- LinearRegression
- LogisticRegression
- DecisionTreeClassifier
- RandomForestClassifier
- KNeighborsClassifier

Handling Missing Data

Missing data occurs when one or more values in a dataset are unavailable, empty, or recorded as null. It is a common issue in real-world Machine Learning because datasets collected from surveys, sensors, transactions, applications, or other sources may not always be complete.

Most Machine Learning algorithms require properly prepared input data. Therefore, missing values need to be handled during the **data preprocessing** stage.

There are three common approaches to dealing with missing values:

1. Drop Rows with Missing Values

Rows containing missing values can simply be removed from the dataset.

- Easy to implement.
- Useful when only a very small number of rows contain missing values.
- Preserves the remaining columns.
- However, removing too many rows can significantly reduce the amount of training data.
- Important information contained in those rows may also be lost.

2. Drop the Entire Column

If a particular feature contains a very large proportion of missing values, the entire feature can be removed.

- Simple solution.
- Eliminates the missing-value problem for that feature.
- However, useful information may be lost.
- Removing an important feature can negatively affect model performance.

3. Impute Missing Values

Imputation means replacing missing values with an estimated or predefined value instead of deleting the corresponding data.

This approach is often preferred because it allows us to retain both the **rows and features**.

For numerical features, common choices include:

- Mean
- Median

- Most frequent value
- A fixed constant

Scikit-Learn provides the **SimpleImputer** class to perform this task efficiently.

SimpleImputer

SimpleImputer is a preprocessing tool from Scikit-Learn that replaces missing values according to a selected strategy.

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy="median")
```

strategy="median" tells the imputer to replace missing values with the **median of the corresponding feature**.

The imputer calculates a separate value for each feature rather than using one common value for the entire dataset.

For example, if a numerical dataset contains features such as:

- monthly_salary
- years_experience
- credit_score

the imputer calculates a separate median for each feature.

Selecting Numerical Features

Before using median imputation, we generally work with numerical features because the median is a numerical statistical measure.

A dataset can contain different types of columns, such as numerical and categorical data. Numerical columns can be selected using:

```
employee_num = employee.select_dtypes(include=[np.number])
```

This creates a dataset containing only numerical features.

Fitting the Imputer

After creating the imputer, it needs to **learn the required replacement values** from the data.

```
imputer.fit_transform(employee_num)
```

Imputation Strategies

SimpleImputer supports several strategies for replacing missing values.

1. Mean Strategy

```
imputer = SimpleImputer(strategy="mean")
```

The **mean** is the arithmetic average of the available values in a feature.

For example, if a feature contains:

20, 30, 40, 50

the mean is:

$(20 + 30 + 40 + 50) / 4 = 35$

Missing values can therefore be replaced with 35.

Advantages

- Easy to calculate.
- Commonly used for numerical data.
- Works well when the data is reasonably balanced.

Disadvantage

The mean can be strongly influenced by **outliers**.

2. Median Strategy

```
imputer = SimpleImputer(strategy="median")
```

The **median** is the middle value after the observations have been arranged in sorted order.

For example:

12, 18, 25, 31, 40

The median is:

25

Median imputation is often useful when numerical data contains extreme values because the median is generally less sensitive to outliers than the mean.

Advantages

- More resistant to extreme values.
- Suitable for many numerical features.
- Simple to understand and implement.

Limitation

The median is primarily appropriate for numerical data.

3. Most Frequent Strategy

```
imputer = SimpleImputer(strategy="most_frequent")
```

This replaces missing values with the **most frequently occurring value** in the corresponding feature.

For example, if a categorical feature contains:

Gold, Silver, Gold, Bronze, Gold, Silver

then Gold is the most frequent value.

This strategy is useful for:

- Categorical features
- Discrete numerical features
- Situations where the most common category is a reasonable replacement

Unlike mean or median, the most frequent strategy can be applied to categorical data.

4. Constant Strategy

```
imputer = SimpleImputer(  
    strategy="constant",  
    fill_value="Unknown"
```

)

The constant strategy replaces every missing value with a **specified fixed value**.

The value is provided using fill_value.

For numerical data, a value such as 0 could be used:

```
imputer = SimpleImputer(
```

```
    strategy="constant",
```

```
    fill_value=0
```

```
)
```

For categorical data, a value such as "Unknown" can be appropriate.

Advantages

- Gives complete control over the replacement value.
- Can be useful when the absence of a value has a specific meaning.
- Can work with both numerical and categorical data when an appropriate constant is selected.

Important: Preventing Data Leakage

One of the most important rules when handling missing data is to **fit the imputer only on the training data**.

Suppose we divide our data into:

Training Data

Test Data

The correct process is:

Training Data



Fit Imputer



Transform Training Data

Then:

Test Data



Use Already-Fitted Imputer



Transform Test Data

We should **not** calculate a new median or mean from the test data.

Why?

If information from the test set is used during training or preprocessing, it can cause **data leakage**.

Data leakage means that information that should be unavailable during model training indirectly influences the learning process.

This can produce an unrealistically good evaluation result because the model's preprocessing has already been influenced by information from the test set.

Feature Scaling

Feature Scaling is a data preprocessing technique used to bring numerical features to comparable scales.

In a Machine Learning dataset, different features may have very different ranges. For example, one feature might contain values between 1 and 10, while another might contain values in the thousands. If these features are given directly to certain algorithms, the feature with larger numerical values can have an unnecessarily large influence.

Scaling transforms the numerical values while generally preserving the useful relationships within each feature.

There are two important scaling techniques:

1. **Min-Max Scaling (Normalization)**
2. **Standardization (Z-score Scaling)**

1. Min-Max Scaling

Min-Max Scaling transforms values into a specified range. The most common range is **[0, 1]**, although another range can be selected.

The general formula is:

$$\text{scaled value} = (x - \text{min}) / (\text{max} - \text{min})$$

Where:

- x = original value
- min = minimum value of the feature
- max = maximum value of the feature
- scaled value = transformed value

How it works

Suppose a feature has values ranging from 20 to 80, and we want to scale a value of 50 into the range [0, 1].

$$\begin{aligned}\text{scaled value} &= (50 - 20) / (80 - 20) \\ &= 30 / 60\end{aligned}$$

= 0.5

Therefore, the value 50 becomes 0.5.

The minimum value becomes 0, and the maximum value becomes 1.

Min-Max Scaling with Scikit-Learn

Scikit-Learn provides the MinMaxScaler class.

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
X_scaled = scaler.fit_transform(X_train)
```

By default, MinMaxScaler uses:

[0, 1]

as the target range.

A different range can be specified using `feature_range`.

For example:

```
scaler = MinMaxScaler(feature_range=(-1, 1))
```

This transforms the features into the range:

[-1, 1]

Important point

The scaling range is **not the same thing as the original feature range**.

The scaler learns the minimum and maximum from the training data and then maps those values to the specified target range.

Why Use Min-Max Scaling?

Min-Max Scaling is useful when we want features to have a **fixed and bounded range**.

It can be especially useful for algorithms that work better when input values are within a limited range.

For example, some **neural-network applications** can benefit from appropriately bounded input values.

Advantages

- Produces a clearly defined range.
- Preserves the relative distribution of values.
- Easy to understand and calculate.
- Useful when a bounded input range is desirable.

Disadvantage: Sensitive to Outliers

Min-Max Scaling is **sensitive to outliers**.

Suppose most values of a feature are between 10 and 20, but one extreme value is 1000.

The minimum and maximum are used in the scaling formula, so the extreme value affects the entire transformation.

As a result, most ordinary observations may become compressed into a relatively small portion of the [0,1] range.

2. Standardization

Standardization transforms a feature so that it is centered around zero and expressed in terms of its standard deviation.

The formula is:

standardized value = $(x - \text{mean}) / \text{standard deviation}$

Where:

- x = original value
- mean = mean of the feature
- standard deviation = standard deviation of the feature

- standardized value = transformed value

Understanding Standardization

Suppose a feature has:

Mean = 50

Standard deviation = 10

If an observation has a value of 70:

$$\begin{aligned}\text{standardized value} &= (70 - 50) / 10 \\ &= 2\end{aligned}$$

The resulting value is 2.

This means the original observation is **two standard deviations above the mean**.

Similarly:

- 0 → approximately at the mean
- Positive value → above the mean
- Negative value → below the mean

Unlike Min-Max Scaling, standardization does **not** force values into a fixed range such as [0,1].

A standardized feature can contain values such as:

-2.1, -0.5, 0, 1.3, 3.7

and potentially much larger or smaller values depending on the data.

Standardization with Scikit-Learn

Scikit-Learn provides the StandardScaler class.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X_train)
```

Properties of Standardized Data

After standardization, a feature generally has:

Zero Mean

The average of the transformed feature is approximately:

0

Unit Variance

The variance is approximately:

1

Therefore, standardization is commonly described as producing:

Zero mean and unit variance.

Is Standardization Completely Unaffected by Outliers?

No.

This is an important distinction.

Standardization is generally **less sensitive to outliers than Min-Max Scaling**, but it is **not immune to outliers**.

Why?

Because standardization uses the:

- Mean
- Standard deviation

Both can be affected by extreme observations.

Therefore, an extreme value can still influence the resulting standardized values.

So it is better to say:

Standardization is often less affected by outliers than Min-Max Scaling, but it is not completely robust to outliers.

When Should You Use Which?

There is **no universal rule** that standardization is always better.

The appropriate technique depends on the Machine Learning algorithm and the characteristics of the data.

Min-Max Scaling can be useful when:

- A fixed range is desirable.
- The algorithm benefits from bounded inputs.
- The data does not contain problematic outliers.
- Neural-network inputs need to be appropriately scaled.

Standardization can be useful when:

- Features have very different scales.
- The algorithm relies on distances or optimization.
- You want features centered around zero.
- You are using algorithms that commonly work well with standardized inputs.
- Gradient-based optimization is involved.

Standardization is commonly a good **general-purpose choice**, but it should not be treated as automatically correct for every problem.

Why Scaling Helps Machine Learning

Consider two features:

Feature A → values around 1–10

Feature B → values around 10,000–50,000

Without scaling, some algorithms may be strongly influenced by the numerical magnitude of Feature B.

After scaling:

Feature A → comparable scale

Feature B → comparable scale

This can help algorithms treat the features more appropriately.

Scaling is particularly important for algorithms involving:

- **Distance calculations**
- **Gradient descent**
- **Optimization**
- **Similarity measurements**

Examples include:

- K-Nearest Neighbors
- K-Means
- Support Vector Machines
- Neural Networks
- Logistic Regression
- Linear Regression with certain optimization methods

Tree-based algorithms generally have much less need for feature scaling because their decision rules are based on feature thresholds rather than distances or feature magnitudes.

Avoiding Data Leakage During Scaling

Scaling follows the same important rule as imputation:

Fit the scaler only on the training data.

Correct:

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Incorrect:

```
X_test_scaled = scaler.fit_transform(X_test)
```

The second approach causes the test data to determine its own scaling parameters.

The test set should be treated as unseen data during training and preprocessing.

Constructing Pipelines in Scikit-Learn

A **Pipeline** in Scikit-Learn is a way to combine multiple Machine Learning preprocessing and modeling steps into a **single sequential workflow**.

Instead of performing every preprocessing operation separately, a pipeline allows us to connect them together. The output of one step automatically becomes the input of the next step.

A typical Machine Learning workflow may involve:

Raw Data



Handle Missing Values



Scale Features



Train Model



Make Predictions

A pipeline allows these steps to be defined together.

Why Do We Need Pipelines?

Without a pipeline, preprocessing can become repetitive and error-prone.

For example, suppose we need to:

1. Replace missing values.
2. Scale numerical features.
3. Train a Machine Learning model.

We could perform each operation separately:

```
imputer.fit(X_train)
```

```
X_train = imputer.transform(X_train)
```

```
scaler.fit(X_train)
```

```
X_train = scaler.transform(X_train)
```

```
model.fit(X_train, y_train)
```

Then, when working with test data, we must remember to apply the **same fitted preprocessing objects** in the correct order:

```
X_test = imputer.transform(X_test)
```

```
X_test = scaler.transform(X_test)
```

```
predictions = model.predict(X_test)
```

This works, but there are several opportunities for mistakes.

A pipeline combines these operations into one object.

Creating a Pipeline

Scikit-Learn provides the Pipeline class:

```
from sklearn.pipeline import Pipeline
```

We can then define a sequence of steps:

```
pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler()),  
    ("model", SomeModel())  
])
```

Each step is represented as a **name-object pair**:

```
("step_name", object)
```

For example:

```
("imputer", SimpleImputer(...))
```

```
("scaler", StandardScaler())
```

```
("model", SomeModel())
```

The names are used to identify the individual steps.

Building a Preprocessing Pipeline

Suppose we want to handle missing values and then standardize numerical features.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

numerical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

The output from the imputer automatically becomes the input to the scaler.

Applying the Pipeline

Once the pipeline has been created, we can fit and transform the training data:

```
X_train_prepared = numerical_pipeline.fit_transform(X_train)
```

Here, `fit_transform()` performs the necessary operations sequentially.

Applying the Pipeline to Test Data

After the pipeline has been fitted using the training data, we should use the **same fitted pipeline** to transform test data:

```
X_test_prepared = numerical_pipeline.transform(X_test)
```

Why Pipelines Help Prevent Data Leakage

Pipeline usage helps enforce the correct preprocessing sequence.

Adding a Machine Learning Model to the Pipeline

A pipeline does not have to contain only preprocessing steps. We can also place the Machine Learning model at the end.

For example:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

model_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("regressor", LinearRegression())
])
```

We can train the entire pipeline with:

```
model_pipeline.fit(X_train, y_train)
```

The pipeline performs the steps in order.

Making Predictions with the Pipeline

Once the pipeline has been fitted:

```
predictions = model_pipeline.predict(X_test)
```

The pipeline automatically:

1. Applies the imputation step.
2. Applies the scaling step.
3. Sends the transformed data to the model.
4. Produces predictions.

You don't have to manually call `transform()` on every preprocessing object.

fit() and predict() with a Complete Pipeline

When the pipeline contains a model, we can simply write:

```
pipeline.fit(X_train, y_train)
```

and then:

```
pipeline.predict(X_test)
```

The pipeline handles the preprocessing automatically.

This means the code becomes much cleaner:

Important Rules for Pipeline Steps

A Scikit-Learn pipeline has some important requirements.

Intermediate steps

Steps before the final step must be **transformers**.

For example:

Imputer → Transformer

Scaler → Transformer

They need to be capable of transforming data.

Final step

The final step can be a **predictor/model**.

For example:

LinearRegression

DecisionTreeClassifier

Pipeline with Only Preprocessing

A pipeline can also contain only preprocessing steps:

```
preprocessing_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
])
```

In this case, the pipeline produces transformed data rather than predictions.

```
X_prepared = preprocessing_pipeline.fit_transform(X_train)
```

This is useful when preprocessing needs to be separated from model training.

Pipeline with Preprocessing + Model

Alternatively, we can include the model:

```
full_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler()),  
    ("model", LinearRegression())  
])
```

Then:

```
full_pipeline.fit(X_train, y_train)
```

and:

```
predictions = full_pipeline.predict(X_test)
```

This is often the cleaner approach because preprocessing and prediction are treated as one complete workflow.

Advantages of Pipelines

1. Cleaner Code

Instead of writing many preprocessing commands separately, all operations are defined in one place.

2. Consistency

The same preprocessing sequence is applied whenever the pipeline is used.

3. Reduces Data Leakage

The pipeline can ensure that preprocessing parameters are learned during fitting and reused during transformation.

4. Fewer Mistakes

You don't have to manually remember the correct order of every preprocessing operation.

5. Easy Model Training

Once the pipeline is created:

```
pipeline.fit(X_train, y_train)
```

can perform the complete training workflow.

6. Easy Prediction

Similarly:

```
pipeline.predict(X_test)
```

can preprocess the input and make predictions.

7. Works Well with Cross-Validation

Pipelines are particularly important when using techniques such as **cross-validation** and **hyperparameter tuning** because preprocessing can be performed correctly within each training fold.

Pipeline and Cross-Validation

One major advantage of pipelines is that they work naturally with Scikit-Learn's model evaluation tools.

Suppose we want to perform cross-validation.

Instead of preprocessing the entire dataset first and then performing cross-validation, the pipeline can ensure that preprocessing is fitted separately within each training portion.

Conceptually:

Fold 1

Training portion → Fit preprocessing → Transform → Train model

Validation portion → Transform using learned preprocessing → Evaluate

Fold 2

Training portion → Fit preprocessing → Transform → Train model

Validation portion → Transform using learned preprocessing → Evaluate

This helps prevent information from the validation portion from leaking into preprocessing.

Accessing Pipeline Steps

Individual steps can be accessed by their names.

For example:

```
pipeline.named_steps
```

This allows us to inspect the components of the pipeline.

A particular step can be accessed using its name:

```
pipeline.named_steps["scaler"]
```

This can be useful when we need to inspect the learned parameters of a preprocessing step.

Pipeline as a Single Estimator

One of the most useful ideas is that the **entire pipeline behaves like a single Scikit-Learn estimator**.

For example:

```
pipeline.fit(X_train, y_train)
```

trains the entire workflow.

And:

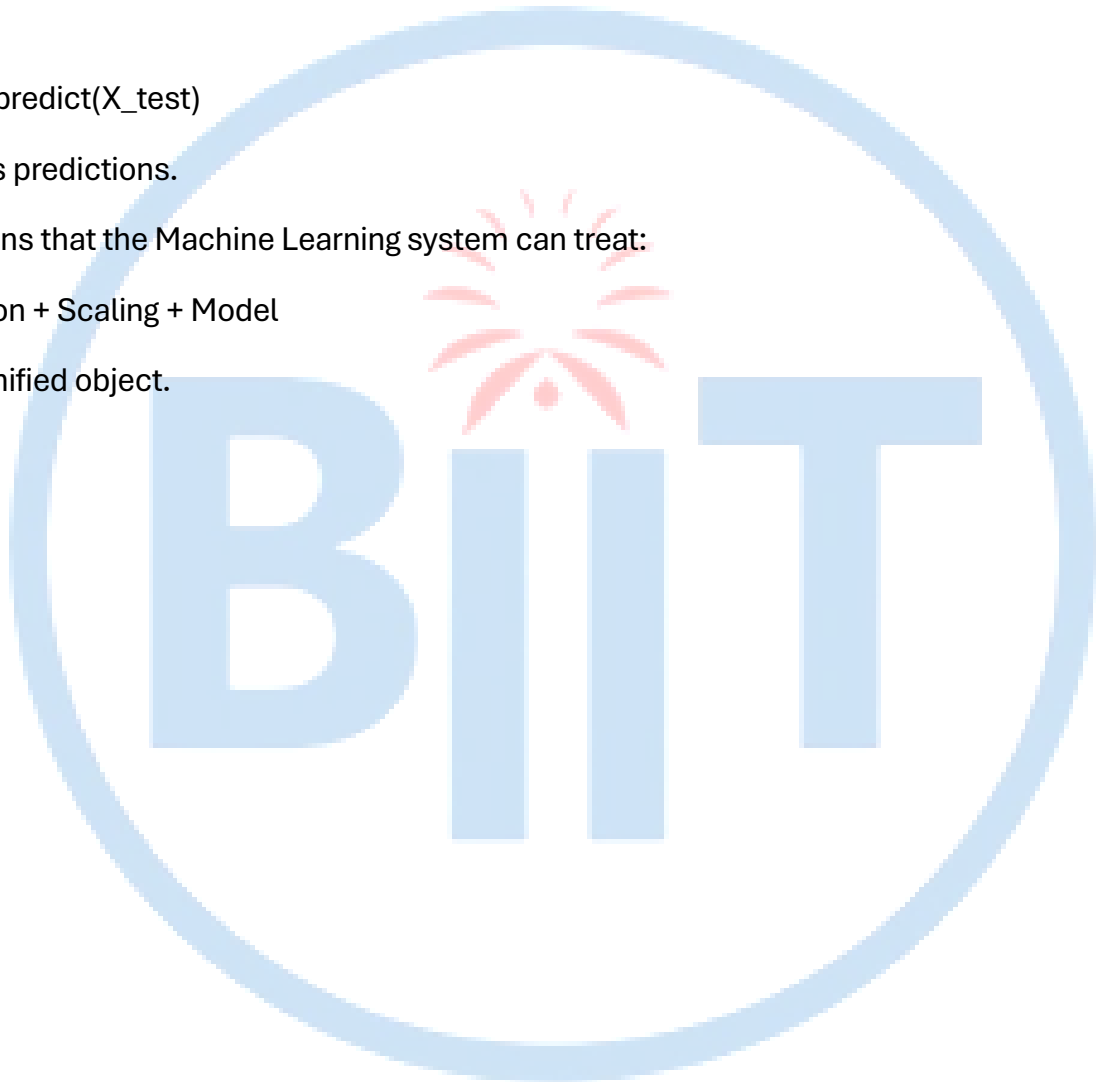
```
pipeline.predict(X_test)
```

produces predictions.

This means that the Machine Learning system can treat:

Imputation + Scaling + Model

as one unified object.



ColumnTransformer

In a real Machine Learning dataset, different columns often require **different preprocessing techniques**.

For example, a dataset may contain:

- Numerical features → missing values need to be imputed and values need to be scaled.
- Categorical features → missing values may need to be imputed and categories need to be converted into numbers.
- Some columns → may need to be left unchanged or removed.

Applying the same transformation to every column would therefore be inappropriate.

ColumnTransformer in Scikit-Learn solves this problem by allowing us to apply **different preprocessing pipelines to different groups of columns**.

Why Do We Need ColumnTransformer?

Suppose our dataset contains these types of features:

Numerical:

- age
- annual_income
- account_balance

Categorical:

- city
- membership_type

We might want to perform:

Numerical Columns



Imputation



Standardization

Categorical Columns



Imputation



One-Hot Encoding

Without ColumnTransformer, we would have to manually separate the columns, preprocess them independently, and then combine them again.

ColumnTransformer handles this process automatically.

What is ColumnTransformer?

ColumnTransformer is a Scikit-Learn utility that allows us to apply **different transformers to different columns** of a dataset.

It is imported using:

```
from sklearn.compose import ColumnTransformer
```

Its basic structure is:

```
ColumnTransformer([  
    ("name1", transformer1, columns1),  
    ("name2", transformer2, columns2)  
])
```

Each tuple contains:

(name, transformer, columns)

name

A name used to identify the transformation.

transformer

The preprocessing operation or pipeline that should be applied.

columns

The columns to which that transformer should be applied.

Example

Suppose we have:

```
num_features = ["age", "salary"]
```

```
cat_features = ["city", "department"]
```

We can create separate preprocessing pipelines:

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
num_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
])
```

```
cat_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="most_frequent")),  
    ("encoder", OneHotEncoder(handle_unknown="ignore"))  
])
```

])

Then combine them using ColumnTransformer:

```
from sklearn.compose import ColumnTransformer  
  
preprocessing = ColumnTransformer([  
    ("num", num_pipeline, num_features),  
    ("cat", cat_pipeline, cat_features)  
)
```

Applying ColumnTransformer

Once the ColumnTransformer has been created, we can fit and transform the training data:

```
X_train_prepared = preprocessing.fit_transform(X_train)
```

During fit_transform():

1. Numerical columns are passed to the numerical pipeline.
2. Missing numerical values are handled.
3. Numerical values are scaled.
4. Categorical columns are passed to the categorical pipeline.
5. Missing categorical values are handled.
6. Categorical values are encoded.
7. The resulting transformed columns are combined into one output.

The result is a single transformed feature matrix.

Applying It to Test Data

After fitting the transformer on training data:

```
X_test_prepared = preprocessing.transform(X_test)
```

The already-learned preprocessing parameters are used.

We **do not** fit the transformer again on the test data.

Different Ways to Specify Columns

ColumnTransformer allows columns to be selected in different ways.

Using column names

```
num_features = ["age", "salary"]
```

```
cat_features = ["city", "department"]
```

This is convenient when working with a Pandas DataFrame.

Using column positions

Columns can also be selected using their numerical positions.

For example:

```
[0, 1, 4]
```

would refer to columns at those positions.

Using selectors

Scikit-Learn also provides tools such as:

```
from sklearn.compose import make_column_selector
```

This can be useful when columns need to be selected based on their data types.

For example, numerical columns can be selected automatically:

```
make_column_selector(dtype_include=np.number)
```

and categorical columns can similarly be selected according to their data type.

Remainder Parameter

What happens to columns that are **not included** in the specified transformations?

By default, ColumnTransformer drops them.

For example:

```
ColumnTransformer([
```

```
("num", num_pipeline, num_features),  
("cat", cat_pipeline, cat_features)  
)
```

Any columns not included in num_features or cat_features are normally dropped.

We can change this behavior using:

```
remainder="passthrough"
```

For example:

```
preprocessing = ColumnTransformer(  
    [  
        ("num", num_pipeline, num_features),  
        ("cat", cat_pipeline, cat_features)  
    ],  
    remainder="passthrough"  
)
```

Now unspecified columns are passed through **without transformation**.

Advantages of ColumnTransformer

1. Different Preprocessing for Different Columns

Numerical and categorical features can be processed appropriately.

2. Cleaner Code

There is no need to manually separate, transform, and recombine every group of columns.

3. Consistent Preprocessing

The same transformations are automatically applied to new data.

4. Reduces Data Leakage

When used properly within a pipeline, preprocessing is learned only from the relevant training data.

5. Works with Cross-Validation

It integrates naturally with Scikit-Learn's model-selection tools.

6. Works with Pipelines

Complex preprocessing workflows can be organized into reusable components.

7. Handles Mixed-Type Data

It is particularly useful for datasets containing both **numerical and categorical features**.

Cross-Validation

Cross-validation is a model evaluation technique used to estimate how well a Machine Learning model will perform on **unseen data**.

Instead of relying on a single validation split, cross-validation evaluates the model **multiple times using different portions of the training data**. This generally gives us a more reliable estimate of the model's generalization performance.

Why Do We Need Cross-Validation?

Suppose we divide our dataset into:

Training Data → 80%

Test Data → 20%

The test set is kept aside for the final evaluation.

K-Fold Cross-Validation

The most common form is **K-Fold Cross-Validation**.

The training data is divided into **K approximately equal-sized folds**.

For example, with:

$cv=5$

we have:

Fold 1

Fold 2

Fold 3

Fold 4

Fold 5

The model is trained and validated five times.

Round 1

Training: Fold 2 + Fold 3 + Fold 4 + Fold 5

Validation: Fold 1

Round 2

Training: Fold 1 + Fold 3 + Fold 4 + Fold 5

Validation: Fold 2

Round 3

Training: Fold 1 + Fold 2 + Fold 4 + Fold 5

Validation: Fold 3

Round 4

Training: Fold 1 + Fold 2 + Fold 3 + Fold 5

Validation: Fold 4

Round 5

Training: Fold 1 + Fold 2 + Fold 3 + Fold 4

Validation: Fold 5

Thus, **every fold is used as a validation set exactly once.**

Why Is This Better?

With a single validation split, the model's score depends on that particular split.

With K-Fold Cross-Validation:

- The model is evaluated multiple times.
- Different observations are used for validation.

- Every observation gets an opportunity to be part of validation.
- We can calculate an average score.
- We can observe how much the score varies between folds.

Therefore, cross-validation usually gives us a **more robust estimate of model performance**.

cross_val_score()

Scikit-Learn provides the convenient function:

```
from sklearn.model_selection import cross_val_score
```

We can use it like this:

```
scores = cross_val_score(  
    model,  
    X_train,  
    y_train,  
    cv=5  
)
```

Here:

- model → Machine Learning estimator
- X_train → training features
- y_train → target values
- cv=5 → 5-fold cross-validation
- scores → score obtained from each fold

For example:

```
print(scores)
```

could produce:

```
[0.86 0.89 0.87 0.91 0.88]
```

The actual values depend on the dataset and model.

Mean Cross-Validation Score

We usually calculate the average of the fold scores:

```
scores.mean()
```

For example:

Scores:

0.86

0.89

0.87

0.91

0.88

The mean score summarizes the overall performance across the folds.

Mean CV Score ≈ 0.882

So we might report:

The model achieved an average cross-validation accuracy of approximately 88.2%.

Standard Deviation

The mean tells us the **average performance**, but we can also calculate the standard deviation:

```
scores.std()
```

The standard deviation tells us how much the scores vary between folds.

For example:

Fold Scores:

0.87

0.88

0.86

0.89

0.88

The scores are relatively close together, indicating stable performance.

But if we had:

0.95

0.72

0.91

0.68

0.94

the large variation would indicate that model performance is much less consistent across folds.

Therefore:

Choosing the Number of Folds

The number of folds is controlled using cv.

cv=5

means 5-fold cross-validation.

cv=10

means 10-fold cross-validation.

With 10-fold cross-validation:

9 folds → Training

1 fold → Validation

This process is repeated 10 times.

Larger K

A larger number of folds generally means:

- More training rounds
- More computational cost
- More data used for training in each round

Smaller K

A smaller number of folds generally means:

- Fewer training rounds
- Lower computational cost
- Each validation fold contains more observations

There is no universally best value. **5 or 10 folds are common choices.**

Cross-validation is performed on the **training data**.

The test set remains untouched.

The test set is used **later**, after we have selected and trained the final model.

Why Keep the Test Set Separate?

Suppose we are comparing three models:

Model A → CV Score = 84%

Model B → CV Score = 89%

Model C → CV Score = 86%

We select Model B based on its cross-validation performance.

Only after making our modeling decisions do we evaluate the final model on the test set.

This gives us a more trustworthy estimate of how the final model performs on genuinely unseen data.

Cross-Validation with a Pipeline

Cross-validation becomes especially important when preprocessing is involved.

```
pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler()),  
    ("model", LogisticRegression())
```

])

Then:

```
scores = cross_val_score(  
    pipeline,  
    X_train,  
    y_train,  
    cv=5  
)
```

Now Scikit-Learn handles preprocessing separately within each fold.

Cross-Validation for Classification

For classification problems, we often want each fold to contain a similar proportion of the different classes.

For example, suppose:

Class 0 → 80%

Class 1 → 20%

We don't want one fold to accidentally contain a very different class distribution.

Scikit-Learn can use **Stratified K-Fold Cross-Validation** for classification.

```
from sklearn.model_selection import StratifiedKFold
```

```
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

Then:

```
scores = cross_val_score(
```

```
model,  
X_train,  
y_train,  
cv=cv  
)
```

Stratification attempts to preserve the class proportions across folds.

Cross-Validation for Regression

For regression problems, ordinary K-Fold Cross-Validation is commonly used:

```
scores = cross_val_score(  
    model,  
    X_train,  
    y_train,  
    cv=5,  
    scoring="neg_mean_squared_error"  
)
```

The appropriate scoring metric depends on the problem.

For example:

Classification

```
scoring="accuracy"
```

Regression

```
scoring="neg_mean_squared_error"
```

or:

```
scoring="neg_mean_absolute_error"
```

Important Note About Negative Error Scores

Scikit-Learn uses a **"higher is better"** convention for scoring functions.

Therefore, error metrics such as Mean Squared Error are returned as negative values when used with `cross_val_score()`:

```
scoring="neg_mean_squared_error"
```

For example:

```
[-120.5, -135.2, -110.7, -128.4, -117.9]
```

A value closer to zero is better because the underlying MSE is smaller.

If we want the actual positive MSE:

```
mse_scores = -scores
```

Cross-Validation for Model Comparison

Cross-validation can be used to compare multiple models.

For example:

```
linear_scores = cross_val_score(  
    linear_model,  
    X_train,  
    y_train,  
    cv=5  
)
```

```
tree_scores = cross_val_score(  
    tree_model,  
    X_train,  
    y_train,  
    cv=5  
)
```

Then compare:

```
print(linear_scores.mean())
```

```
print(tree_scores.mean())
```

The model with better cross-validation performance may be preferred, although other factors should also be considered.

Advantages of Cross-Validation

Better use of training data

Instead of permanently setting aside a large validation portion, different parts of the training data can be used for validation at different times.

More reliable estimate

Multiple validation scores provide more information than a single validation score.

Model comparison

Different algorithms can be compared using the same cross-validation strategy.

Hyperparameter tuning

Cross-validation provides a way to evaluate different hyperparameter configurations.

Detecting instability

Large variation between fold scores may indicate that the model's performance is sensitive to the particular training/validation split.

Disadvantages

Cross-validation is not free.

If we use:

```
cv=10
```

the model needs to be trained approximately 10 times.

Therefore:

More folds, More training runs and More computation

For computationally expensive models, cross-validation can become costly.

Complete Example Structure

A common Scikit-Learn architecture is:

```
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_features),
    ("cat", cat_pipeline, cat_features)
])

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("model", SomeModel())
])
```

Then:

```
full_pipeline.fit(X_train, y_train)
```

and:

```
predictions = full_pipeline.predict(X_test)
```

The entire process is handled automatically.

Complete ML Model

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
```

```
# Load Dataset
```

```
df = pd.read_csv("customer_data.csv")
```

```
# Separate Features and Target
```

```
X = df.drop("renewed", axis=1)
```

```
y = df["renewed"]
```

```
# Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Define Numerical and Categorical Columns
```

```
num_features = [
```

```
    "age",
```

```
    "monthly_spend",
```

```
    "support_calls",
```

```
    "months_active"
```

```
]
```

```
cat_features = [
```

```
    "plan_type",
```

```
    "region",
```

```
"payment_method"  
]
```

```
# Numerical Preprocessing Pipeline
```

```
num_pipeline = Pipeline(  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
)
```

```
# Categorical Preprocessing Pipeline
```

```
cat_pipeline = Pipeline(  
    ("imputer", SimpleImputer(strategy="most_frequent")),  
    ("encoder", OneHotEncoder(handle_unknown="ignore"))  
)
```

```
# Column Transformer
```

```
preprocessing = ColumnTransformer(  
    ("num", num_pipeline, num_features),  
    ("cat", cat_pipeline, cat_features)  
)
```

```
# Complete ML Pipeline
```

```
full_pipeline = Pipeline(  
    ("preprocessing", preprocessing),
```

```
("model", LogisticRegression(max_iter=1000))  
])
```

```
# Cross-Validation
```

```
cv_scores = cross_val_score(  
    full_pipeline,  
    X_train,  
    y_train,  
    cv=5,  
    scoring="accuracy"  
)
```

```
print("Cross-Validation Scores:", cv_scores)  
print("Mean CV Score:", cv_scores.mean())  
print("CV Standard Deviation:", cv_scores.std())
```

```
# Train Final Model
```

```
full_pipeline.fit(X_train, y_train)
```

```
# Make Predictions
```

```
y_pred = full_pipeline.predict(X_test)
```

```
# Evaluate Model
```

```
print("\nTest Accuracy:")
```

```
print(accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```



Model Persistence

Model persistence means saving a trained machine learning model so that it can be reused later without training it again.

This approach saves time and computational resources because the model does not need to be retrained every time predictions are required. It also helps maintain consistency and reproducibility.

Why Train and Save?

Training a machine learning model can be computationally expensive and time-consuming. Instead of repeating the training process, the trained model can be saved and reused whenever needed.

- The trained model is saved as model.pkl.
- The preprocessing pipeline is saved as pipeline.pkl.
- The saved files can be loaded later for prediction.
- Retraining is avoided, making future predictions faster.
- It improves reproducibility because the same trained model can be reused.

Why Use Joblib?

Joblib is a Python library commonly used for saving and loading machine learning models and other Python objects.

It is particularly useful with scikit-learn because many machine learning models contain large numerical arrays.

- Efficiently saves trained scikit-learn models.
- Handles large NumPy-based data efficiently.
- Allows models and preprocessing pipelines to be loaded later.
- Makes model deployment and reuse easier.

The program checks whether a previously trained model already exists. This determines whether it needs to train a new model or directly perform inference.

Model does not exist: Train the model, create the preprocessing pipeline, and save them.

Model already exists: Load the saved model and pipeline and directly make predictions.

For Example:

```
import os

import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import StratifiedShuffleSplit
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestRegressor

def create_preprocessor(numerical_columns, categorical_columns):
    numerical_pipeline = Pipeline([
        ("fill_missing", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])
    categorical_pipeline = Pipeline([
        ("encode", OneHotEncoder(handle_unknown="ignore"))
    ])
```

```
return ColumnTransformer([  
    ("numerical", numerical_pipeline, numerical_columns),  
    ("categorical", categorical_pipeline, categorical_columns)  
])
```

```
if os.path.exists("model.pkl") and os.path.isfile("pipeline.pkl"):
```

```
# Load previously trained model and pipeline  
regressor = joblib.load(model.pkl)  
preprocessor = joblib.load(pipeline.pkl)  
  
new_data = pd.read_csv("input.csv")  
  
processed_data = preprocessor.transform(new_data)  
  
predicted_values = regressor.predict(processed_data)  
  
new_data["predicted_house_value"] = predicted_values  
  
new_data.to_csv("output.csv", index=False)  
  
print("Saved model used for prediction.")
```

```
else:
```

```
# Load dataset

_data = pd.read_csv("data.csv")

# Create categories for stratified sampling
_data["group"] = pd.cut(
    _data["income "],
    bins=[0, 1.5, 3, 4.5, 6, np.inf],
    labels=[1, 2, 3, 4, 5]
)

# Create stratified training split
splitter = StratifiedShuffleSplit(
    n_splits=1,
    test_size=0.2,
    random_state=42
)

train_indices, _ = next(
    splitter.split(_data, _data["group"])
)

training_data = _data.iloc[train_indices].copy()

# Remove temporary category
training_data.drop("group", axis=1, inplace=True)
```

```
# Separate target and input features

target = training_data.pop("value")

features = training_data

# Identify numerical and categorical columns

categorical_columns = ["col"]

numerical_columns = [
    column for column in features.columns
    if column not in categorical_columns
]

# Create and apply preprocessing pipeline
preprocessor = create_preprocessor(
    numerical_columns,
    categorical_columns
)

prepared_features = preprocessor.fit_transform(features)

# Create and train Random Forest model
regressor = RandomForestRegressor(
    random_state=42
)

regressor.fit(prepared_features, target)
```

```
# Save trained objects

joblib.dump(regressor, "model.pkl")

joblib.dump(preprocessor, "pipeline.pkl")


print("Model trained")
```

Hyper Parameter Tuning : GridSearchCV

GridSearchCV is a hyperparameter-tuning technique provided by scikit-learn.

It automatically searches through different combinations of hyperparameters and determines which combination gives the best cross-validation performance.

Import:

```
from sklearn.model_selection import GridSearchCV
```

Example:

Suppose you have an SVM:

```
SVC()
```

and you don't know the best values for:

C

kernel

gamma

You can specify:

```
param_grid = {  
    'C': [0.1, 1, 10],  
    'kernel': ['linear', 'rbf'],  
    'gamma': ['scale', 'auto']  
}
```

GridSearchCV tries every possible combination.

Number of combinations:

$$3 \times 2 \times 2 = 12$$

If cv=5, each combination is evaluated using 5-fold cross-validation:

$$12 \text{ combinations} \times 5 \text{ folds} = 60 \text{ model fits}$$

Why do we need GridSearchCV?

Machine-learning models have two types of parameters:

Model parameters

These are learned from the training data.

For example, in linear regression:

weights

intercept

You don't normally choose these manually.

Hyperparameters

These are set before training.

Examples:

Random Forest:

n_estimators
max_depth
min_samples_split

SVM:

C
kernel
gamma

KNN:

n_neighbors
weights
metric

Logistic Regression:

C
penalty

The best hyperparameter values depend on the dataset.

GridSearchCV helps find good values automatically.

Syntax:

```
GridSearchCV(  
    estimator,  
    param_grid,  
    scoring=None,  
    n_jobs=None,  
    refit=True,  
    cv=None,  
    verbose=0,  
    pre_dispatch='2*n_jobs',  
    error_score=np.nan,  
    return_train_score=False  
)
```

The most important parameters are:

estimator
param_grid
scoring
cv
n_jobs
verbose
refit

estimator

The model you want to tune.

Example:

```
from sklearn.ensemble import RandomForestClassifier  
  
model = RandomForestClassifier()  
  
grid = GridSearchCV(  
    estimator=model,  
    param_grid=param_grid  
)
```

Other examples:

```
GridSearchCV(SVC(), param_grid)
```

```
GridSearchCV(  
    LogisticRegression(),  
    param_grid  
)
```

```
GridSearchCV(  
    RandomForestClassifier(),  
    param_grid  
)
```

param_grid

This specifies the hyperparameters and the values you want to test.

It is usually a Python dictionary.

Example:

```
param_grid = {  
    'n_estimators': [50, 100, 200],
```

```
'max_depth': [None, 5, 10],  
'criterion': ['gini', 'entropy']  
}
```

GridSearchCV tries every combination.

Number of combinations:

$$3 \times 3 \times 2 = 18$$

With:

cv=5

total model fits:

$$18 \times 5 = 90$$

Multiple parameter grids

You can also provide a list of dictionaries.

Example:

```
param_grid = [  
    {  
        'kernel': ['linear'],  
        'C': [0.1, 1, 10]  
    },  
    {  
        'kernel': ['rbf'],  
        'C': [0.1, 1, 10],  
        'gamma': ['scale', 'auto']  
    }  
]
```

This is useful when different parameters apply to different model configurations.

scoring

This is one of the most important parameters.

It tells GridSearchCV how to decide which model is best.

Example:

```
GridSearchCV(  
    model,  
    param_grid,  
    scoring='accuracy'  
)
```

You can use different scoring metrics depending on your problem.

Classification scoring values

Common classification scoring values include:

Accuracy

```
scoring='accuracy'
```

Formula:

Accuracy = Correct predictions / Total predictions

Good when classes are reasonably balanced.

Balanced accuracy

```
scoring='balanced_accuracy'
```

Useful when classes are imbalanced.

It calculates the average recall across classes.

Precision

scoring='precision'

Precision:

$$TP / (TP + FP)$$

Recall

scoring='recall'

Recall:

$$TP / (TP + FN)$$

F1 score

scoring='f1'

F1 is the harmonic mean of precision and recall.

$$F1 = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Useful when you care about both precision and recall.

F-beta

scoring='fbeta'

More commonly, use:

ROC AUC

scoring='roc_auc'

Useful for binary classification when the model provides scores/probabilities.

ROC AUC — One-vs-One

`scoring='roc_auc_ovo'`

Used for multiclass classification.

ROC AUC — One-vs-Rest

`scoring='roc_auc_ovr'`

Also used for multiclass classification.

Average Precision

`scoring='average_precision'`

Particularly useful for imbalanced classification problems.

Log Loss

`scoring='neg_log_loss'`

Important:

The name contains **neg** because scikit-learn's scoring system is designed so that higher scores are better.

Actual log loss:

lower is better

So scikit-learn uses:

negative log loss

where:

higher = better

Regression scoring values

For regression, common values are:

R^2

scoring='r2'

Higher is better.

Typical interpretation:

1 → perfect

0 → roughly as good as predicting the mean

< 0 → worse than the mean baseline

Mean Absolute Error

scoring='neg_mean_absolute_error'

MAE:

average |actual - predicted|

Normally:

lower = better

But GridSearchCV expects higher scores to be better, so scikit-learn uses:

negative MAE

Therefore:

-2 > -5

means the model with -2 is better.

Mean Squared Error

scoring='neg_mean_squared_error'

MSE:

average(actual - predicted)²

Lower actual MSE is better.

GridSearchCV therefore uses:

negative MSE

Root Mean Squared Error

scoring='neg_root_mean_squared_error'

RMSE:

sqrt(MSE)

Again, because lower is better, scikit-learn uses the negative version.

Mean Absolute Percentage Error

scoring='neg_mean_absolute_percentage_error'

Median Absolute Error

scoring='neg_median_absolute_error'

Explained Variance

scoring='explained_variance'

Higher is better.

How to see all available scoring metrics

Instead of memorizing every metric, use:

```
from sklearn.metrics import get_scorer_names  
  
print(get_scorer_names())
```

This gives the scoring names supported by your installed version of scikit-learn.

This is the best way to check the exact list because scoring options can change between scikit-learn versions.

cv

cv means cross-validation.

Example:

cv=5

means 5-fold cross-validation.

The training data is divided into 5 parts:

Fold 1 → validation

Fold 2 → validation

Fold 3 → validation

Fold 4 → validation

Fold 5 → validation

Each fold gets used as validation once.

Conceptually:

Round 1:

Train Train Train Train | Validation

Round 2:

Train Train Train | Validation | Train

Round 3:

Train Train | Validation | Train Train

Round 4:

Train | Validation | Train Train Train

Round 5:

Validation | Train Train Train Train

Examples:

```
from sklearn.model_selection import KFold  
  
cv = KFold(n_splits=5, shuffle=True, random_state=42)
```

Then:

```
GridSearchCV(  
    model,  
    param_grid,
```

```
CV=CV  
)  
  
For classification:  
  
from sklearn.model_selection import StratifiedKFold  
  
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

StratifiedKFold tries to preserve class proportions in each fold.

n_jobs

n_jobs controls how many CPU jobs/processes can be used in parallel.

This is particularly important because GridSearchCV may perform many model fits.

n_jobs=None

Default behavior.

Generally means:

Use 1 job

unless another parallelism context is active.

n_jobs=1

Use one CPU worker.

n_jobs=-1

Use all available CPU processors/workers.

This is extremely common.

```
GridSearchCV(  
    model,  
    param_grid,  
    n_jobs=-1  
)
```

verbose

verbose controls how much information GridSearchCV prints while running.

verbose=0 (Default)

No progress messages.

verbose=1

Print basic progress information.

verbose=2

Print more detailed information.

verbose=3

Print even more detailed progress information, including more timing/details.

Higher values

Higher positive values can produce increasingly detailed output depending on the underlying joblib/scikit-learn operation.

Example:

```
GridSearchCV(  
    model,  
    param_grid,  
    cv=5,  
    verbose=2  
)
```

refit

This is another important parameter.

Default:

refit=True

After GridSearchCV identifies the best hyperparameters, it fits the best model again using the complete training dataset.

Example:

```
grid = GridSearchCV(  
    model,  
    param_grid,  
    cv=5,  
    refit=True  
)
```

Then:

grid.best_estimator_

is available.

And you can directly do:

```
grid.predict(X_test)
```

refit=False

GridSearchCV will search for the best parameters but will not automatically refit the best estimator on the whole training data.

This can be useful when you don't need the refitted model.

Multiple scoring metrics

You don't have to use only one scoring metric.

Example:

```
scoring = {  
    'accuracy': 'accuracy',  
    'precision': 'precision',  
    'recall': 'recall',  
    'f1': 'f1'  
}
```

Then:

```
grid = GridSearchCV(  
    model,  
    param_grid,  
    scoring=scoring,  
    refit='f1',  
    cv=5  
)
```

Now you can inspect:

`grid.cv_results_`

and compare all metrics.

return_train_score

Default:

`return_train_score=False`

If you set:

`return_train_score=True`

GridSearchCV also stores training scores.

Example:

```
grid = GridSearchCV(  
    model,  
    param_grid,  
    cv=5,  
    return_train_score=True  
)
```

Then:

`grid.cv_results_['mean_train_score']`

can be inspected.

This can help identify overfitting.

For example:

Training score = 0.99

Validation score = 0.72

This suggests possible overfitting.

error_score

Controls what happens if fitting a particular hyperparameter combination produces an error.

Default is:

```
error_score=np.nan
```

For example:

```
GridSearchCV(  
    model,  
    param_grid,  
    error_score=np.nan  
)
```

If one combination fails, its score can be set to NaN and the search can continue.

You can also use:

```
error_score='raise'
```

Then the error is raised immediately.

This is very useful while debugging.

pre_dispatch

Controls how many jobs are dispatched ahead of execution when parallel processing is used.

Default:

```
pre_dispatch='2*n_jobs'
```

For example:

```
n_jobs=4
```

then:

```
pre_dispatch = 8
```

You can also specify:

```
pre_dispatch=4
```

or:

```
pre_dispatch='all'
```

This can matter when model fitting is memory-intensive.

Complete example

```
from sklearn.model_selection import GridSearchCV  
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(random_state=42)
```

```
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [None, 5, 10],  
    'min_samples_split': [2, 5]  
}
```

```
grid = GridSearchCV(  
    estimator=model,  
    param_grid=param_grid,  
    scoring='accuracy',  
    cv=5,  
    n_jobs=-1,  
    verbose=2,  
    refit=True,
```

```
return_train_score=True  
)
```

```
grid.fit(X_train, y_train)
```

Getting the best parameters

```
grid.best_params_
```

Example:

```
{  
    'max_depth': 10,  
    'min_samples_split': 2,  
    'n_estimators': 200  
}
```

This means GridSearchCV found this combination to be best according to the specified scoring metric.

Getting the best score

```
grid.best_score_
```

Example:

0.93

This is the best mean cross-validation score.

Important:

best_score_ is not your test-set score.

It is the best CV score on the training data used during the search.

Getting the best model

```
grid.best_estimator_
```

Example:

```
best_model = grid.best_estimator_
```

Then:

```
y_pred = best_model.predict(X_test)
```

Test-set evaluation

After hyperparameter tuning:

```
from sklearn.metrics import accuracy_score
```

```
y_pred = grid.predict(X_test)
```

```
print(accuracy_score(y_test, y_pred))
```

cv_results_

This is extremely useful.

```
grid.cv_results_
```

It contains detailed results for every parameter combination.

You can convert it to a DataFrame:

```
import pandas as pd
```

```
results = pd.DataFrame(grid.cv_results_)
```

```
print(results)
```

EXAMPLE:

```
from sklearn import datasets

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.svm import SVC

# Load dataset

iris = datasets.load_iris()

X, y = iris.data, iris.target

# Split data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model

svm = SVC()

# Parameter grid

param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

# Grid Search

grid = GridSearchCV(
```

```
estimator=svm,  
param_grid=param_grid,  
cv=5,  
scoring='accuracy',  
n_jobs=-1  
)
```

```
grid.fit(X_train, y_train)
```

```
print("Best Parameters:", grid.best_params_)
```

```
print("Best Score:", grid.best_score_)
```

```
# Evaluate on test set
```

```
test_accuracy = grid.score(X_test, y_test)
```

```
print("Test Accuracy:", test_accuracy)
```

RandomizedSearchCV

RandomizedSearchCV is a hyperparameter tuning technique in scikit-learn. It searches for the best hyperparameter combination by randomly selecting a fixed number of combinations instead of trying every possible combination.

It is especially useful when the hyperparameter search space is large and GridSearchCV would take too much time.

```
from sklearn.model_selection import RandomizedSearchCV
```

Why RandomizedSearchCV

Suppose you have:

```
param_distributions = {  
    'n_estimators': [50, 100, 200, 300, 500],  
    'max_depth': [5, 10, 20, 30, None],  
    'min_samples_split': [2, 5, 10, 20],  
    'max_features': ['sqrt', 'log2', None]  
}
```

There are:

$5 \times 5 \times 4 \times 3 = 300$ combinations

With $cv=5$, GridSearchCV would perform:

$300 \times 5 = 1500$ model fits

With RandomizedSearchCV:

$n_iter=30$

it tests only 30 randomly selected combinations:

$30 \times 5 = 150$ model fits

So it can save a lot of computation.

Basic Syntax

```
random_search = RandomizedSearchCV(  
    estimator=model,  
    param_distributions=param_distributions,  
    n_iter=20,  
    scoring='accuracy',  
    cv=5,  
    n_jobs=-1,  
    verbose=1,  
    random_state=42  
)
```

Then:

```
random_search.fit(X_train, y_train)
```

estimator

estimator is the machine-learning model whose hyperparameters you want to tune.

Examples:

```
SVC()
```

```
RandomForestClassifier()
```

```
RandomForestRegressor()
```

```
LogisticRegression()
```

```
GradientBoostingClassifier()
```

Example:

```
model = RandomForestClassifier()
random_search = RandomizedSearchCV(
    estimator=model,
    ...
)
```

param_distributions

param_distributions contains the hyperparameters and the values or distributions from which RandomizedSearchCV will sample.

```
param_distributions = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [5, 10, 20, None],
    'min_samples_split': [2, 5, 10]
}
```

Unlike GridSearchCV, RandomizedSearchCV can also work with probability distributions.

For example:

```
from scipy.stats import randint
param_distributions = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(5, 50)
}
```

This allows it to randomly select integer values within the specified ranges.

n_iter

n_iter specifies how many parameter combinations should be sampled.

n_iter=10

means 10 random combinations.

For example:

```
RandomizedSearchCV(  
    model,  
    param_distributions,  
    n_iter=30,  
    cv=5  
)
```

The approximate number of model fits is:

30 combinations × 5 folds = 150 fits

This is one of the most important parameters in RandomizedSearchCV.

scoring

scoring tells RandomizedSearchCV how to evaluate each model.

For classification, common options are:

scoring='accuracy'

scoring='precision'

scoring='recall'

scoring='f1'

scoring='roc_auc'

scoring='balanced_accuracy'

For regression:

```
scoring='r2'
```

```
scoring='neg_mean_absolute_error'
```

```
scoring='neg_mean_squared_error'
```

```
scoring='neg_root_mean_squared_error'
```

You can see the scoring names available in your installed scikit-learn version using:

```
from sklearn.metrics import get_scorer_names  
print(get_scorer_names())
```

cv

cv specifies the cross-validation strategy.

```
cv=5
```

means 5-fold cross-validation.

If:

```
n_iter=20
```

```
cv=5
```

then:

$20 \times 5 = 100$ model fits

You can also provide a CV splitter:

```
from sklearn.model_selection import StratifiedKFold
```

```
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42
```

)

Then:

RandomizedSearchCV(

model,

param_distributions,

cv=cv

)

n_jobs

n_jobs controls how many CPU workers are used for parallel processing.

Common values:

n_jobs=None

Default.

Generally uses one worker unless controlled by another parallelism context.\

n_jobs=1

Uses one worker.

n_jobs=2

Uses up to two workers.

n_jobs=-1

Uses all available CPU workers.

Very commonly used with RandomizedSearchCV.

Example:

```
RandomizedSearchCV(  
    model,  
    param_distributions,  
    n_jobs=-1  
)
```

verbose

verbose controls how much progress information is printed.

verbose=0

No progress information.

Default.

verbose=1

Basic progress information.

verbose=2

More detailed information.

verbose=3

Even more detailed output.

random_state

random_state controls the random selection of hyperparameters.

random_state=42

Using a fixed value makes the random search reproducible.

For example:

```
random_search = RandomizedSearchCV(  
    model,  
    param_distributions,  
    n_iter=30,  
    random_state=42  
)
```

42 has no special meaning. You can use other integers:

random_state=0

random_state=10

random_state=100

The important thing is using a fixed value when you want reproducible results.

refit

refit=True is the default.

After finding the best hyperparameters, RandomizedSearchCV refits the best model using the complete training data.

refit=True

Then you can directly use:

```
random_search.predict(X_test)
```

If:

```
refit=False
```

the best estimator isn't automatically refitted.

you can access the results using:

```
random_search.best_params_
```

Gives the best hyperparameter combination.

```
random_search.best_score_
```

Gives the best mean cross-validation score.

```
random_search.best_estimator_
```

Gives the best fitted model.

```
random_search.cv_results_
```

Contains detailed results for all sampled combinations.

You can convert the results into a DataFrame

EXAMPLE

```
from sklearn.model_selection import RandomizedSearchCV  
from sklearn.ensemble import RandomForestClassifier  
from scipy.stats import randint
```

```
model = RandomForestClassifier(random_state=42)
```

```
param_distributions = {  
    'n_estimators': randint(50, 500),  
    'max_depth': randint(5, 50),  
    'min_samples_split': randint(2, 20),  
    'min_samples_leaf': randint(1, 10),  
    'max_features': ['sqrt', 'log2', None]  
}
```

```
random_search = RandomizedSearchCV(  
    estimator=model,  
    param_distributions=param_distributions,  
    n_iter=30,  
    scoring='accuracy',  
    cv=5,  
    n_jobs=-1,  
    verbose=1,  
    random_state=42  
)
```

```
random_search.fit(X_train, y_train)
```

```
print(random_search.best_params_)
```

```
print(random_search.best_score_)
```

```
best_model = random_search.best_estimator_
```

```
y_pred = best_model.predict(X_test)
```

